第 1 讲练习答案：整数与汇编入门

ICS 第一次小测复习 · 独立练习答案 · 请先完成题目再核对

对应第 1 讲正文。这是教学练习解析，不是用户作答记录。首次独立完成 D1–P1—P7 后再核对；途中得到提示的作答点不计入首次独立正确率，但可以记录修正后的理解。

必做题合计 **25 分，20 分达到 80% 门槛**。各题为 P1 4 分、P2 3 分、P3 3 分、P4 4 分、P5 3 分、P6 3 分、P7 5 分。以下评分是本讲训练口径，不是任何原卷的官方评分。小问含多个要求时按列明评分点给分，不用“整题全对才得分”。

D1–P1：两种解释与字节序（4 分）

1. 二进制是 `1011 0110`。（1 分）

一位十六进制换四位：B 对应 1011，6 对应 0110。此时只是在写位模式，还没决定正负。

2. 无符号为 182，8 位补码为 −74。（各 1 分）

无符号值：11×16+6=182。最高位是 1，按补码解释需减去 2^8，得到 182−256=−74。也可按负权重算 $-128+32+16+4+2=-74$。

3. 小端内存排列如下。（1 分）

地址	0x5000	0x5001	0x5002	0x5003
字节内容	D4	C3	A2	81

先把完整数拆成 81、A2、C3、D4 四个字节；小端将最低有效字节 D4 放到最低地址。不翻转单个字节内的位。四格均正确得 1 分；若整体方向反了，回看正文图 D1–F3。

D1–P2：位运算、逻辑与短路（3 分）

1. `a & b=0`，`a && b=1`。（各 0.5 分）

```
a = 0x28 = 0010 1000
b = 0x14 = 0001 0100
按位与       0000 0000
```

两个数中为 1 的位没有重合，所以按位与为 0。但 a、b 都不是零，分别当逻辑真，因此逻辑与为 1。

2. 不执行除法，r=0。（各 0.5 分）

`z!=0` 为假；`&&` 在左边为假时短路，不再计算 `18/z>1`。不能说“先除零，再因为左边为假所以没事”；这里安全的原因是右边根本没有被求值。

3. 可写 `x & ~0xFu`，也可写 `x & 0xFFFFFFF0u`。（1 分）

低四位的掩码为 1111，取反后为低四位 0、其余位 1。与 0 会清零，与 1 会保留。本题 unsigned int 是 32 位，因此两个表达式等价。

D1–P3：整数提升与公共类型（3 分）

1. a 的表示为 `0xFFFFFFEE`。（1 分）

−18 的 8 位补码是 256−18=238，即 0xEE。c 扩成 int 时保留数值 −18，所以复制符号位到高位。只写 `0x000000EE` 会把它变成正数 238。

2. 在 int 中相加，t=270。（类型和结果各 0.5 分）

unsigned char 的所有值都能用 32 位 int 表示，所以 p、q 先提升为 int；计算 240+30=270；目标 t 又是 int，不会发生 8 位截断。这题没有“保留低 8 位”的步骤。

3. `-3 < 2U` 为假；`-3L < 2U` 为真。（每个判断及其对应类型合计 0.5 分）

- 第一式：int 与 unsigned int 同等级，公共类型是 unsigned int。−3 转成 $2^{32}-3$，这是大正数，小于 2 不成立。
- 第二式：本环境 long 是有符号 64 位，能够容纳所有 unsigned int，公共类型是 long。因此比较的是 −3L 和 2L，小于成立。

若只写“因为有 U 所以都转无符号”，第二问会出错。要先看类型等级与能否容纳。

D1–P4：右移、bias 与语言规则（4 分）

1. 逻辑右移为 `0010 1011`；算术右移为 `1110 1011`。（各 0.5 分）

两种操作都丢弃最右两位。逻辑右移在左边补两个 0；算术右移复制原最高位 1，所以补两个 1。

2. 除法为 −3，算术右移为 −4；bias=7，修正后为 −3。（四项各 0.5 分）

数学商 −25/8=−3.125。C 整数除法向零截断，因此得到 −3；本题算术右移向下取整，得到 −4。用 $2^3-1=7$ 修正：`-25+7=-18`，`-18` 算术右移 3 位就是 `floor(-18/8)=-3`。

不要把 bias 加到正数上：例如 9/8 应得 1，若对正数也加 7，则 16>>3 会错成 2。这个偏置是专为负数两种舍入方向不同而加入的。

3. 不能保证。该 C 表达式有未定义行为。（1 分）

1U 的类型是 32 位 unsigned int，移位量 32 等于左操作数宽度。C 要求移位量非负且小于该宽度；本题违反条件。不能因为 x86 某条移位指令会处理计数低位，就宣布 C 表达式等于 1。答出“未定义行为”及宽度依据得分；只写“不是 1，而是 0”不得分。

D1–P5：子寄存器（3 分）

三问都从原 rax=`0x89ABCDEFFEDCBA98` 出发，不顺次叠加。

小问	完整 rax	解释
1	`0x89ABCDEFFEDCBA11`	只写低 8 位 al，其余 56 位保留
2	`0x0000000076543210`	写 eax 得到低 32 位，同时清高 32 位
3	`0x00000000FFFFFF98`	原 al=0x98，符号扩展到 eax 得 0xFFFFFF98；写 eax 又清 rax 高位

每项 1 分。第三问不能写 `0xFFFFFFFFFFFFFF98`，那是扩展到 64 位的 `movsbq` 才产生的结果；本题 `movsbl` 目标为 32 位。

如果第三题做错，先单独写 eax 的值，再执行“写 eax 清高位”这一步，不要凭“有符号”三个字一次把 64 位都补成 1。

D1–P6：寻址与传送（3 分）

先算共同的地址表达式：

```
0x10 + 0x200 + 3×8
= 0x10 + 0x200 + 0x18
= 0x228
```

小问	rax 的值	是否读取数据内存	解释
1	`0x228`	否	lea 只保存地址表达式的计算结果
2	`0x8899`	是	movq 从 0x228 开始读取 8 字节
3	`0x228`	否	`$` 表示立即数，直接复制常量

每问结果 0.5 分、是否访存 0.5 分。第一行与第三行结果一样，但来源不同：一个通过算术计算得到，一个来自指令里的立即数。这些回答均不统计 CPU 为执行程序而取指令的访问。

D1–P7：五行状态追踪（5 分）

用 `M32[A]` 表示从地址 A 开始的 4 字节位模式。本题初始 `M32[0x1008]=0x00000007`，`M32[0x100C]=0xFFFFFFFE`。

行	源值/地址计算	该行执行后的改变	内存行为
1	0x1000+2×4=0x1008	rdx=`0x1008`	lea 不读写数据内存
2	读 M32[0x1008]，值为 0x00000007	rax=`0x0000000000000007`	读 4 字节，内存不变
3	地址 rdx+4=0x100C；源位模式 0xFFFFFFFE；原 eax=7	rax=`0x0000000000000005`	读 4 字节，内存不变
4	源 eax=5；目的地址 rdx=0x1008	M32[0x1008]=`0x00000005`	写 4 字节；寄存器不变
5	读 M32[0x100C]，按 32 位补码为 −2	rcx=`0xFFFFFFFFFFFFFFFE`	读 4 字节；内存不变

第三行是 32 位机器加法：`0x00000007 + 0xFFFFFFFE = 0x1_00000005`，只保存低 32 位 0x00000005；写 eax 同时清 rax 高位。也可按补码理解成 7+(−2)=5。不能把进位产生的第 33 位留在 rax 中。

第四行只覆盖 0x1008—0x100B，不会改动 0x100C—0x100F。因此第五行仍然读到原来的 −2；符号扩展的目标直接是 64 位 rcx，高位全补 1。rdi、rsi 和 rdx 在相应行之外均保持不变。

评分：每行 1 分，其中源/有效地址 0.5 分，目的的值及读写方向 0.5 分。 第 2、3 行若只报告 eax 而忽略要求的完整 rax，结果部分最多得 0.25 分；第 4 行若只写“结果为 5”却没说明写到内存，结果部分不得分。最终数值碰巧正确，不能补回错误的内存读写方向。

D1–A1：可选边界题

不纳入必做 25 分。另设 3 个自检评分点，每小问 1 分，结果与原因都正确才通过该点。

1. `-INT_MIN` 不能保证得到正数。

INT_MIN=-2^{31}，对它取数学相反数是 2^{31}，大于 INT_MAX=$2^{31}-1$。结果不能用 int 表示，C 有符号运算溢出，属于未定义行为。若另一个题明确要求 32 位机器取负，则其位模式会保持最小负数的模式；这不是该 C 表达式的普遍语言保证。

2. a+1U 为 0，但 INT_MAX+1 无可靠的 C 数值结果。

a 是 unsigned int：$(2^{32}-1)+1=2^{32}$，对 2^{32} 取模为 0，属于明确规定的无符号运算。INT_MAX+1 在 int 中超过上界，属于有符号溢出。两者碰巧可能在某段低级机器代码中用相同加法器，也不能抹平 C 语义区别。

3. a+1 是 int 类型的 256；赋给 unsigned char b 后 b=0。

unsigned char 的 a 先提升为 int，再和 int 常量 1 相加；256 没超过 int 范围。赋值给 unsigned char 才转换到 8 位无符号，按模 256 得到 0。顺序必须是“提升 → 加法 → 转窄”，不是“8 位加法自然变零”。

完成检查：能分别指出未定义行为、无符号取模与整数提升三个不同原因，才算掌握这道边界题。

D1–A2：可选对齐题

改编来源：2025 期末第一题第 3 小题，PDF 第 2 页；移位量由 5 改为 3。本题不纳入必做 25 分。另设 3 个自检评分点：等价掩码 1 分、用商余数说明大小和差值范围 1 分、数值验证及倍数情形 1 分。

1. 等价表达式为 x & ~7U，也可写 x & 0xFFFFFFF8U。

右移 3 位丢掉低 3 位；再左移 3 位让保留下来的位回到原位置，但最右边只能补 0。两步合起来就是清低 3 位。

2. y≤x，且 0≤x−y≤7。

把无符号整数写成 $x=8q+r$，其中 $0\leqslant r\leqslant 7$。右移得到 q，再左移得到 8q，因此 $y=8q=x-r$。删掉的 r 非负，所以 $y\leqslant x$；差恰好是 r，因此最多是 7。

这里的左移不会额外丢失本来应保留的有效位：第一步右移已把高端空出三位，第二步只是把这些保留下来的位移回去。不能单凭“任何左移都可能溢出”而拒绝分析这个组合。

3. x=77 时 y=72，差为 5；x 为 8 的倍数时 y=x。

$77=8\times 9+5$；右移得 9，左移得 72。8 的倍数余数 r=0，即低 3 位本来全为 0，清零不改变它。这就是“向下对齐到 8 的倍数”的位操作含义。

核对后如何决定下一步

若 P1/P3 错，回到位模式、数值与转换顺序；若 P2 错，单独区分逐位与逻辑真假；若 P4 错，先分清机器和 C，再处理取整；若 P5 错，分成“指令目标宽度”和“写 32 位清高位”两步；若 P6/P7 错，逐行画出有效地址与目的位置。

达到 20/25 后，也应确保你能独立说清有符号与无符号比较、正确画一次小端排列，并在 P7 中区分读内存和写内存。没有达标时修错因，再用未见小变式复测；重抄本答案不算独立通过。此处不填写任何尚未发生的分数或学习表现。

第 2 讲练习答案：浮点与条件分支

ICS 第一次小测复习 · 独立练习答案 · 请先完成题目再核对

对应第 2 讲讲义与题面。请先完成独立作答再核对。D2–P01—D2–P08 为必做，共 30 分；首次独立作答达到 **24/30（80%）** 为本讲门槛。得到提示后完成的部分属于练习结果，不回填成首次独立得分。两道选做各附 5 分参考评分，但不计入 30 分，也不能用来抵消必做题遗漏。

评分按下列作答点给分；分数用于定位薄弱处，并非原考试官方评分。所有独立题均为助手补充，未使用 2025 第一次小测的具体题目。

题号	必做分值	主要检查点
D2–P01	4	分段、分类、解码公式
D2–P02	4	边界与特殊值
D2–P03	4	中点舍入与重新规格化
D2–P04	3	数值转换、精度、向零截断
D2–P05	4	CF 与 OF 的区别
D2–P06	3	比较顺序、signed/unsigned
D2–P07	3	部分寄存器与标志保持
D2–P08	5	分支路径与条件传送限制
合计	**30**	**24 分为 80%**

D2–P01　解码两个不同类别的数（4 分）

本题 $k=4, n=3$，Bias=$2^3-1=7$。

`0x25`：

1. `0x25 = 00100101₂ = 0 | 0100 | 101`。
2. $s=0$，$e=4$，$F=5$；阶码既非全 0，也非全 1，是规格化数。
3. $E=4-7=-3$，$M=1+5/8=13/8$。
4. $V=(13/8)2^{-3}=\boxed{13/64}$。

`0x83`：

1. `0x83 = 10000011₂ = 1 | 0000 | 011`。
2. $s=1$，$e=0$，$F=3\neq 0$，是负的非规格化数。
3. $E=1-7=-6$，$M=3/8$，不能添隐藏的 1。
4. $V=-(3/8)2^{-6}=\boxed{-3/512}$。

评分：每个数 2 分，分段与类别 0.5 分，E、M 同时正确 0.5 分，最后数值 1 分。若第二问使用 $E=-7$ 或 $M=1+3/8$，分别属于“非规格化指数”和“隐藏位”错误，不是单纯算术失误。

D2–P02　范围与特殊值（4 分）

$k=3, n=3$，Bias=3，最小规格化指数 $E_{\min}=1-3=-2$，最大有限指数 $E_{\max}=6-3=3$。

所问	字段	计算	答案
最小正非规格化数	`0|000|001`	$2^{-3}\times 2^{-2}$	$\boxed{1/32}$
最小正规格化数	`0|001|000`	1×2^{-2}	$\boxed{1/4}$
最大有限正数	`0|110|111`	$(1+7/8)\times 2^{3}$	$\boxed{15}$
`0|111|010`	阶码全 1，frac 非零	不套有限数公式	$\boxed{\mathrm{NaN}}$

评分：每项 1 分。前三项公式或字段依据 0.5 分、数值 0.5 分；最后一项类别 0.5 分、阶码与 frac 的判断依据 0.5 分。把最后一个数认成正无穷，是遗漏了“无穷还要求 frac 全 0”。

D2-P03　中点与重新规格化（4 分）

本题格式 `1/4/3`，Bias=7。

$21/4=5.25$：

$$5.25=101.01_2=1.0101_2\times 2^2。$$

保留三位 frac 后是 `1.010`，删掉的部分为 `1000...`，正好在 5 与 5.5 中间。保留末位是 0，所以不进位，舍成 $1.010_2\times 2^2=5$。

$e=2+7=9=1001_2$，故结果为

```
0 | 1001 | 010 = 01001010₂ = 0x4A
```

$31/8=3.875$：

$$3.875=11.111_2=1.1111_2\times 2^1。$$

保留 `1.111`，删掉的也是中点模式 `1000...`，但保留末位为 1。这次必须进位成 `10.000₂×2¹`，重新规格化得到 `1.000₂×2²=4`。

$e=2+7=9=1001_2$，故结果为

```
0 | 1001 | 000 = 01001000₂ = 0x48
```

评分：每问 2 分。精确二进制式 0.5 分，舍入判据与方向 0.5 分，最终数值 0.5 分，编码 0.5 分。第二问若写出进位但仍保留 E=1，则最后数值和编码相应扣分；这一处需要回看“重新规格化”，不必重学整套 IEEE 格式。

D2-P04　数值转换（3 分）

1. **成立。** binary64 有 53 位有效精度，足以精确保存所有 32 位 int。转换为 double 没丢数值，再转回 int 也不会越界。因此 `x == (int)(double)x` 为真。结论 0.5 分、精度与范围理由 0.5 分。
2. $16777219=2^{24}+3$。binary32 在这个区间的间距为 2，邻近刻度是 $2^{24}+2=16777218$ 与 $2^{24}+4=16777220$，原数恰在中间。后者保留有效数的最低位为 0，所以舍成 16777220。再转回 int 不含小数也不越界，答案为 $\boxed{16777220}$。数值 0.5 分，间距、中点与偶数判据 0.5 分。这里“偶数”看目标格式的末位；两个十进制候选本身都是偶数，不能靠十进制奇偶选。
3. `(int)-6.75` = $\boxed{-6}$，浮点到 int 在可表示范围内向零截断。数值 0.5 分、方向 0.5 分。回答 −7 是把向下取整混入了 C 转换。

D2-P05　分别计算 CF 和 OF（4 分）

片段	al 的最终值	CF	ZF	SF	OF
A：`0x7f + 1`	`0x80`	0	0	1	1
B：`0x00 − 1`	`0xff`	1	0	1	0

A 的理由：无符号 127+1=128，没有超出 8 位无符号范围，CF=0；补码的 127+1 超出最大值 127，结果位串为 `10000000`，表示 −128，OF=1。该位串非零、最高位为 1，所以 ZF=0、SF=1。

B 的理由：无符号 0 减 1 需要借位，CF=1；补码 0−1=−1 在范围内，OF=0。截断后的位串全 1，非零且最高位是 1，所以 ZF=0、SF=1。

评分：每段 2 分。al 0.5 分，CF 0.5 分，OF 0.5 分，ZF 与 SF 合计 0.5 分。只把溢出标志都写成 1，说明仍未把无符号范围与补码范围分开。

D2-P06　选择 signed 与 unsigned 条件（3 分）

`cmpl %esi,%edi` 依据 **edi−esi** 设置标志，不改 edi、esi。

- 按补码解释：edi=−1，esi=1，−1<1，所以 `setl %al` 写入 $\boxed{1}$。
- 按无符号解释：edi=$2^{32}-1$，esi=1，前者并不小于后者，所以 `setb %al` 写入 $\boxed{0}$。

若用标志核对，32 位差为 `0xfffffffe`，CF=0、ZF=0、SF=1、OF=0。于是 signed 小于条件 SF≠OF 成立，unsigned 小于条件 CF 不成立。set 指令不修改这些标志，两次查看同一比较结果没有冲突。

评分：两个 set 结果各 1 分；解释比较次序并区分两种数值解释 1 分。仅说“setl 看有符号，setb 看无符号”而没有本题比较依据，解释项给 0.5 分。

D2-P07　部分寄存器与条件码保持（3 分）

`testl %edi,%edi` 的按位与结果是 0，所以 CF=0、ZF=1、SF=0、OF=0。它不修改 edi，也不修改 rax。

`setne` 条件是 ZF=0。本题条件不成立，因此只把 al 改为 0。rax 的其余 56 位保持原值：

```
执行 setne 后：rax = 0xaabbccddeeff0000
```

`movzbl %al,%eax` 将 0 零扩展到 32 位，写 eax 又清零 rax 的高 32 位：

```
执行 movzbl 后：rax = 0x0000000000000000
```

两条指令都不改条件码，因此最终仍是 $\boxed{CF=0,\ ZF=1,\ SF=0,\ OF=0}$。在 setne 后高位仍有非零旧值，不能把整个 rax 当作整数 0。

评分：setne 后的完整 rax 1 分；movzbl 后的完整 rax 1 分；四个标志与保持原因合计 1 分。若只写 al=0，第一项只给 0.5 分，因为本题特意要求区分部分寄存器与完整寄存器。

D2-P08　短分支的 C 与汇编对照（5 分）

`cmpq %rsi,%rdi` 比较 x−y；`jg .Lelse` 在 signed x>y 时跳到 else。因此不跳的路径是 x≤y，C 条件为：

```
long choose(long x, long y) {
    if (x <= y) return x + 2;
    else return y - 1;
}
```

两个输入的追踪如下：

x,y	比较与跳转	执行的表达式	返回值
−3,2	x>y 为假，不跳	x+2	−1
7,2	x>y 为真，跳到 `.Lelse`	y−1	1

改成 unsigned long 比较时，需要在 unsigned x>y 时跳到 else，因此把 `jg` 换成 $\boxed{ja}$。题目仅改变比较类型；无符号加减本身按无符号运算规则解释。

不能把任意 `p ? *p : 0` 都改成先读 `*p` 再条件传送。当 p 为空时，原表达式不会解引用，提前读取却已经触发非法访问；后来选择 0 不能撤销这个访问。

评分：C 条件 1 分；两次返回值各 1 分；`ja` 1 分；指出提前解引用空指针的风险 1 分。最后一项只说“cmov 不一定好”没有说明原因不给分。

D2–P09　选做：同样位数，不同有限实数数量（参考 5 分）

阶码全 1 的模式都留给无穷或 NaN，不属于有限值。其余 exp 一共有 2^k-1 种，每个 exp 有 2^n 种 frac，再乘两个符号，即有限位模式数量为：

$$2(2^k-1)2^n。$$

这些有限模式中，+0 与 −0 数值相同；其余正负有限数各有唯一模式。因此不同有限实数数量为 $2(2^k-1)2^n-1$。

- A：$2\times 7\times 8-1=\boxed{111}$。
- B：$2\times 15\times 4-1=\boxed{119}$。

最小正非规格化数是 $2^{1-\mathrm{Bias}-n}$：

- A：Bias=3，得到 $2^{-5}=\boxed{1/32}$。
- B：Bias=7，得到 $2^{-8}=\boxed{1/256}$。

B 的 frac 少一位，单个数量级的刻度较稀；但本题中可用阶码更多，且全 1 阶码占用的位模式更少，所以不同有限实数总数反而较多。这并不意味着 B 在每一个数附近都更精确。

参考评分：两种有限数数量各 1 分，说明排除全 1 阶码及两个零去重 1 分，两种最小正数各 1 分。

D2–P10　选做：严格大小关系可能变成相等（参考 5 分）

原整数是 $a=2^{24}$，$b=2^{24}+1$，所以 a<b。

binary32 从 2^{24} 开始相邻刻度间隔为 2。a 可精确表示；b 位于 2^{24} 与 $2^{24}+2$ 的中间，最近偶数舍入选择前者。所以：

```
(float)a = 16777216
(float)b = 16777216
```

浮点转换后的 < 为假，== 为真。若先分别转成 double，两者均可精确表示，仍保持 < 为真、== 为假。

这没有违反非严格单调性。非严格单调性只保证原来 a≤b 时，舍入后仍有 Round(a)≤Round(b)，允许两个不同输入变成相等。它并不保证严格小于关系一直保留。

参考评分：原整数关系 1 分，两个 float 数值和舍入依据 1 分，float 比较结论 1 分，double 结论及精确性 1 分，非严格单调性解释 1 分。

按失分位置安排下一步

如果 P01—P03 丢分，先分清是字段分类、边界还是舍入，不把所有错误统称为“浮点不会”。如果 P05 正确而 P06 错，重点复查 signed/unsigned 的解释和后缀；如果 P07 错，回看部分寄存器写入规则；如果 P08 路径算错，在 cmp 下写出 x−y，再在分支旁写“跳”和“不跳”的条件。

达到 24 分后仍应重做真正不会的点；未达标时先修两个最核心错误，再用未见变式验证。不要反复背同一道题的十六进制答案来替代掌握步骤。这里只提供评判方法，不代填你的成绩、错题或学习状态。

第 3 讲独立练习答案与评分

ICS 第一次小测复习 · 独立练习答案 · 请先完成题目再核对

对应第 3 讲：循环、跳转表与过程调用。建议先完整写出自己的答案，再按小问核对。必做 D3–P01—D3–P04 每题 10 分，共 40 分；32 分为基础过关线。D3–P05 为选做 10 分，单独记录，不补入必做分数。

这里的分值是本套练习的训练评分，不是往年试卷的官方评分。等价 C 写法、不同但正确的推导均可得分；程序行为与边界比变量名字重要。若前一小问发生算术笔误，后续沿用该数但方法独立正确，可保留相应方法分，避免把同一个小错反复扣分。首次独立得分、得到提示后的结果和订正结果分开记录。

D3–P01：恢复循环并核对边界（10 分）

1. 寄存器角色（1 分）

`%rax` 保存累计值 `sum`，`%rdx` 保存当前奇数计数器 `i`，每项 0.5 分。`%rcx` 是每轮临时算出的 `3*i`，知道它有助于理解，但本问不额外计分。

2. 初始化、条件、更新（2 分）

- 初始化 `sum=0,i=1`：0.5 分。
- 条件是 `i<=n`：1 分。`cmpq %rdi,%rdx` 形成 `i-n` 的条件码，`jle` 是有符号“小于或等于”。
- 每轮 `i+=2`：0.5 分。

进入循环前先 `jmp .Lp1_test`，所以第一轮也要先检查条件。把条件写成 `n<=i` 不能得条件分。

3. 等价 C（2 分）

```c
long loop3(long n) {
    long sum = 0;
    for (long i = 1; i <= n; i += 2) {
        sum += 3 * i;
    }
    return sum;
}
```

循环控制完整正确给 1 分，累计 `3*i` 并返回正确变量给 1 分。用 `while` 或 `goto` 的等价实现也可以；不能写成先执行一次的无入口检查 `do-while`，因为这会在 `n=0` 时改变行为。给定 `n<=100`，计算不会发生 `long` 有符号溢出。

4. n=5（1 分）

进入循环体时 i	本轮加入	累计 sum	更新后的 i
1	3	3	3
3	9	12	5
5	15	27	7

加入的数为 `3`、`9`、`15`，给 0.5 分；最终返回 `27`，给 0.5 分。`i=7` 时条件失败，不再进入循环体。

5. n=0（1 分）

不执行循环体，返回零。因为初始 `i=1`，第一次先到测试位置，`1<=0` 不成立。结果正确给 0.5 分，指出入口先检查的原因给 0.5 分。

6. jle 改为 jl（1 分）

条件变成 `i<n`；`n=5` 时只加入 3、9，返回 12。条件变化给 0.5 分，结果给 0.5 分。差别不是机械地“少一轮”对所有输入都成立：当边界 `n` 为偶数时，奇数计数器原本也不会恰好等于它。

7. 循环不变式（2 分）

一种表述是：每次到达测试位置，`i` 为正奇数，`sum` 等于所有小于 `i` 的正奇数分别乘以 3 后的和，即

```
sum = 3 × (1 + 3 + ... + (i−2))
```

`i=1` 时右侧为空和零，初始成立。若当前条件通过，循环体把 `3*i` 加入，随后 `i` 增加 2，所以新 `i` 之前的所有正奇数恰好都被累加，关系继续成立。

正确关系给 1 分，初始与保持理由合计给 1 分。也可用已执行轮数 `k` 表述 $i=1+2k, sum=3k^2$，但必须解释它与执行过程的联系，不能只给一个没有定义 `k` 的公式。

D3–P02：跳转表、默认分支与落入（10 分）

1. 映射（2 分）

x	入口标签
0	`.Lp2_a`
1	`.Lp2_b`
2	`.Lp2_default`
3	`.Lp2_c`
4	`.Lp2_c`

五项全对 2 分；每错一项扣 0.5 分，最低零分。`x=2` 在索引范围内，但它的表项主动指向默认代码，这与范围外直接跳去默认代码是两条不同路径。

2. 负数为何去默认分支（1 分）

`ja` 使用无符号“大于”条件。`-1` 的 64 位位模式按无符号解释是 `2^64-1`，大于 4，因此跳去默认分支。说清无符号比较给 0.5 分，说明负数位模式落在大值区间给 0.5 分。

3. 共享入口与落入（1 分）

`x=3` 和 `x=4` 共用 `.Lp2_c`，0.5 分。`.Lp2_a` 完成 `a+=2` 后直接继续执行 `.Lp2_b`，对应 `case 0` 落入 `case 1`，0.5 分。不能说 0 和 1“完全相同”，因为 0 在到达共同代码前修改了参数。

4. 等价 C（2 分）

```
long route(long x, long a) {
    switch (x) {
    case 0:
        a += 2;
        /* 有意落入 */
    case 1:
        return 5 * a;
    case 3:
    case 4:
        return a - 3;
    default:
        return -1;
    }
}
```

0、1 的落入和结果正确给 1 分，3、4 及默认路径正确给 1 分。`case 2` 可不单独写，因为它与默认路径一致；显式加 `case 2:return -1;` 同样正确。输入范围保证运算不溢出。

5. 三个返回值（1 分）

`route(0,4)=30`，`route(1,4)=20`，`route(3,4)=1`。三项全对 1 分，恰有一项错误给 0.5 分，其余零分。第一个结果需要先把 4 加 2，再乘 5。

6. 表项地址与内容（1 分）

读 `0x5000+4*8=0x5020`，0.5 分。该处存 `.Lp2_c` 的代码地址，不是计算结果 `a-3`，0.5 分。

7. 错误替换（2 分）

不能保证正确执行。`leaq .Lp2_table(,%rdi,8),%rax` 只得到表项的地址，没有读取其中的目标地址；随后的 `jmp *%rax` 将跳到表本身的那一项，把数据位置当代码入口。

指出 `lea` 得到表项地址、缺少一次内存读取给 1 分；说明会跳到表区而不是分支代码给 1 分。正确的两步等价写法可为：

```
movq .Lp2_table(,%rdi,8), %rax
jmp *%rax
```

这里仍按题目提供的绝对地址表模型理解。实际程序若使用位置无关代码，地址形成方式可能不同，但不能省略从表项取得目标的步骤。

D3–P03：局部内存、调用前后与栈恢复（10 分）

1. 分配空间（2 分）

进入时 `%rsp=0x2008`。`pushq %rbx` 先减 8，得到 `0x2000`，并把原值 99 写入 `[0x2000]`。随后减 16，得到 `0x1ff0`。

两个 `%rsp` 各 0.5 分，旧 `%rbx` 的地址 1 分。注意十六进制地址减去十进制 16，就是减去 `0x10`。

2. 参数和对齐（1 分）

`%rdi=0x1ff8`，因为 `lea` 传入 `8(%rsp)` 的地址；`%rsi=5`。两参数正确给 0.5 分。调用前 `%rsp=0x1ff0`，为 16 的倍数，满足本题调用要求，给 0.5 分。

3. 被调用函数入口（2 分）

`call` 再压入 8 字节返回地址，故入口 `%rsp=0x1fe8`，`R1` 位于 `[0x1fe8]`。

```
高地址
0x2008  R0：返回外层的地址
0x2000  99：保存的旧 rbx
0x1ff8  7：局部数据；rdi 指向这里
0x1ff0  已预留，题目未写入，内容未知
0x1fe8  R1：返回 caller 的地址  <- rsp
低地址
```

入口 `%rsp` 与 `R1` 位置正确给 1 分，图中另外三个已知对象的相对位置正确给 1 分。图的上下方向可以相反，但必须标明地址顺序。不能给未初始化槽擅自填零。

4. 返回值（1 分）

`read_add` 读出 7，加上第二参数 5，返回 12；它没有写回内存。`caller` 把保留的 `x=4` 加上，最终返回 16。各 0.5 分。

5. 栈与寄存器恢复（2 分）

`read_add` 的 `ret` 弹出 `R1`，回到 `caller` 时 `%rsp=0x1ff0`。随后 `addq $16` 使它成为 `0x2000`；`popq %rbx` 恢复 99，并使 `%rsp=0x2008`；最后 `caller` 的 `ret` 弹出 `R0`，回到外层时 `%rsp=0x2010`。

`read_add` 返回后的值给 0.5 分，`caller` 最终返回后的值给 1 分，`%rbx=99` 给 0.5 分。最容易错的是把 `0x2008` 当最终答案：它是执行最后 `ret` **之前**的栈顶，不是之后。

6. 用 rdi 保留 x 为什么错误（2 分）

本题中的 `%rdi` 在调用前就已经被 `lea` 改成局部数据地址 `0x1ff8`，所以不是 `x`。即使 `read_add` 本身不修改 `%rdi`，调用后加它也会算出 `12+0x1ff8`，不等于 `x+12`。指出这个具体覆盖给 1 分。

此外，`%rdi` 属调用者保存寄存器，普通被调用函数可以改变它。即使某段代码没有提前覆盖它，也不能仅凭 ABI 假定跨调用后仍保留原值。说明这一点给 1 分。

D3–P04：递归每层保存什么（10 分）

1. 基本情形与递推（2 分）

`F(0)=0`，1 分。`n>0` 时，`lea -1(%rdi,%rdi),%rbx` 算出 `2n-1`，然后调用 `F(n-1)`，所以

```
F(n) = (2n-1) + F(n-1)
```

递推正确给 1 分。`lea` 没有读取该数值所对应的内存。

2. 等价 C（1 分）

```c
long odd_sum(long n) {
    if (n == 0) return 0;
    return (2 * n - 1) + odd_sum(n - 1);
}
```

边界、递归参数与返回表达式完整正确给 1 分。虽然结果等于 `n*n`，本问要解释给定递归，不宜只写闭式函数。

3. odd_sum(3) 的过程（2 分）

向下：`n=3` 保留 5，`n=2` 保留 3，`n=1` 保留 1，`n=0` 直接返回 0。向上：`F(1)=1+0=1`，`F(2)=3+1=4`，`F(3)=5+4=9`。

各层保留值正确给 1 分，各层返回值正确给 1 分。只写最终 9 而无过程最多得到本问 0.5 分。

4. rbx 的保存约定（1 分）

父层把自己的 `2n−1` 放入 `%rbx`，期待子调用按 ABI 保持它。子层要覆盖 `%rbx` 时，先用 `push` 保存进入时的值，结束时 `pop` 恢复，因此返回父层时父层的值仍在。

父层自己也先保存了它的调用者原来的 `%rbx`，最终必须恢复。说清子层保护父层值给 0.5 分，说清当前层也要恢复自己的入层旧值给 0.5 分。仅写“递归自动保存寄存器”不得分。

5. 同时活跃数量（1 分）

最多同时活跃 4 个调用：`3`、`2`、`1`、`0` 四层。尚未配对弹出的非基本层保存操作有 3 个。各 0.5 分。基本层虽然没有执行 `push %rbx`，仍然有返回地址。

6. 栈顶与对齐（2 分）

每下降到下一层，先有当前层 `push %rbx` 减 8，再有 `call` 压返回地址减 8，共减少 16 字节。于是：

层	刚入口 rsp	非基本层 push 后、call 前 rsp
n=3	`0x3008`	`0x3000`
n=2	`0x2ff8`	`0x2ff0`
n=1	`0x2fe8`	`0x2fe0`
n=0	`0x2fd8`	基本情形不调用

每层减 16 的解释给 0.5 分，基本层入口 `0x2fd8` 给 1 分，三个调用前地址均对 16 余 0 的检查给 0.5 分。

也可进一步画出最深处的栈，核对“保存的是哪个层的值”：

```
0x3008  外层返回地址
0x3000  进入最外层前的旧 rbx
0x2ff8  n=2 返回 n=3 的地址
0x2ff0  n=3 保留的值 5，被 n=2 的 push 保存
0x2fe8  n=1 返回 n=2 的地址
0x2fe0  n=2 保留的值 3，被 n=1 的 push 保存
0x2fd8  n=0 返回 n=1 的地址  <- rsp
此刻 rbx 为 n=1 保留的值 1
```

这些内部返回地址在该函数中可以是同一个指令地址，因为每层执行的是同一处递归 `call`。**地址相同不等于栈槽相同**：每次调用都把它压在不同地址，返回时按后进先出逐层弹出。

7. 基本情形不能多 pop（1 分）

基本情形在任何 `push %rbx` 之前就跳走，当前栈顶已经是它的返回地址。如果额外 `pop %rbx`，会把返回地址错误弹入 `%rbx`；之后 `ret` 再把上一个栈槽的数据当目标。指出没有匹配的 `push` 给 0.5 分，说明返回地址被误弹给 0.5 分。

D3–P05：超过六个参数的调用（选做，10 分）

1. 前六个寄存器（1 分）

依次为 `%rdi,%rsi,%rdx,%rcx,%r8,%r9`。顺序完全正确 1 分；只对前四个给 0.5 分。

2. 三个栈顶时刻（2 分）

初始 `0x4000`；压入 8 后 `0x3ff8`；压入 7 后 `0x3ff0`；`call` 压入返回地址后，`eight` 入口为 `0x3fe8`。两次压栈后的值各 0.5 分，入口值 1 分。调用前 `0x3ff0` 仍满足 16 字节对齐。

3. 入口布局（2 分）

```
0x4000  调用者原有数据
0x3ff8  8: 第八参数   = 16(%rsp)
0x3ff0  7: 第七参数   = 8(%rsp)
0x3fe8  返回地址      = 0(%rsp)  <- rsp
```

返回地址位置正确给 1 分，两参数位置和次序正确给 1 分。第七个参数并不是入口的 `0(%rsp)`，这个位置已被 `call` 写入返回地址。

4. 返回值与修改的寄存器（2 分）

返回 `7+8+1=16`，1 分。函数体修改 `%rax`，`ret` 还会推进 `%rsp` 并改变控制流；它没有改写一般被调用者保存寄存器，按正常返回恢复调用者栈位置，1 分。允许把 `%rip` 列为控制转移时改变的指令指针，不强求列出。两条 `addq` 还会更新条件码；这是另一个处理器状态变化，不属于破坏被调用者保存的通用寄存器。

5. 回收参数空间（1 分）

`ret` 只弹出返回地址，不替调用者回收两个栈参数。返回后 `%rsp=0x3ff0`，加 16 后恢复成 `0x4000`。原因和结果各 0.5 分。

6. 交换 push 顺序（2 分）

交换后，第七参数位置读到 8，第八参数位置读到 7，违反了原本希望传入 `(...,7,8)` 的次序，1 分。本函数恰好把两数相加，加法交换后结果不变；如果函数算第七参数减第八参数，结果就会改变。因此这次输出没变不能证明传参顺序正确，1 分。

记录与下一步

项目	自己填写
必做首次独立得分	___ / 40
调用栈能否独立画对	___
主要错误出在循环 / 跳表 / 栈 / 递归哪一处	___
具体错因	___
订正后仍需复测的能力	___
选做成绩（若完成）	___ / 10

达到 32 分并能独立画对栈后，进入下一讲。未达标时先修造成连锁错误的前置步骤：例如把 `cmp` 顺序读对，或只练一次 `call` 压入、`ret` 弹出的 8 字节变化，再回到整题。核对后重新抄同一题的满分答案不算通过新的独立验证。

第4讲练习答案

ICS 第一次小测复习 · 独立练习答案 · 请先完成题目再核对

对应第4讲正文。先做题再看本页。D4–P1—P6各3分，P7、P8各11分，总分40；32分为过关参考。题目均为助手补充，下面是本讲训练评分规则，不是官方标准。

D4–P1

小端是 **0x40032001**，大端是 **0x01200340**，联合大小 **4字节**，每项1分。地址从低到高的字节从未改变；改变的是哪一端被视为整数的高位。

D4–P2

`sizeof(a)=3×4×4=48`； `sizeof(p)=8`； p+1 跨过一行四个 int，增加 **16字节**。每项1分。p 的大小和 p 指向对象的大小是两件事。

D4–P3

p 在 **rdi**，k 在 **rsi**，x 在 **xmm0**，每项1分。double 返回值也在 xmm0。p 虽然指向 double，参数自身仍是地址，不占用浮点参数通道。

D4–P4

依次是 **11**、**12**、**24**，每项1分。

写入总长度是 n+1：n=11 最后写到 B+11；n=12 的终止零写到 B+12；n=24 的终止零写到 B+24。不要把缓冲区容量与距离返回地址的距离混成一个量。

D4–P5

ASLR；canary；返回链使用已有可执行代码，栈只存放数据和下一执行地址。每项1分。说出“不可执行栈只禁止把栈内字节作为指令执行，并不禁止 ret 读取栈中的地址”即得到第三项分数。

D4–P6

进入G1时 rsp=**0x808**；G1弹出9到rax，随后ret进入G2；lea算出3×9，所以rax=**27**；G2的ret消耗0x818槽，转到DONE后rsp=**0x820**。每项1分。

完整rsp变化为：0x800 → 初始ret后0x808 → pop后0x810 → G1的ret后0x818 → G2的ret后0x820。lea不读内存，也不改变rsp。

D4–P7

1. **0x2000+4×(5i+j)**。元素线性编号5i+j得1分，换成字节偏移并加首地址得1分。
2. 线性编号13，字节偏移52=0x34，地址 **0x2034**。过程和结果各1分。
3. a+1为 **0x2014**；sizeof(a)=**60字节**，各1分。一个行指针跨20字节。
4. 下列每行1分，等价实现接受：

```
leaq (%rdi,%rdi,4), %rax
addq %rsi, %rax
movl TAB(,%rax,4), %eax
```

5. rows中存放的是指针，先要读出rows[i]（1分）；各行可以不连续，不能预设统一首地址加固定行宽的布局（1分）。

D4–P8

字段	偏移	大小	结束后的偏移
c	0	1	1
s	2	2	4
n	8	8	16
a	16	12	28

1. **0、2、8、16**，每个0.5分，共2分。
2. 整体对齐8；28向上补到八的倍数，所以 **sizeof=32**，各1分。
3. a[2]在对象起点后 `16+2×4=24=0x18` 字节，地址 **0x3018**。偏移和结果各1分。
4. 第二个对象晚 **32字节**（1分）；结构数组的步长是含内部和尾部填充的sizeof，保证下一个对象成员仍正确对齐（1分）。
5. a起于4，z起于 `align(4+4N,8)`。条件为 `8<4+4N<=16`（1分），所以 **N=2或3**（1分）。N=2时数组结束于12，需要四字节填充；N=3时恰好结束于16。知道z偏移不足以排除前一种情况（1分）。

D4–X1 选做

p在rdi，x在xmm0，y在xmm1，返回值在xmm0。一种示意主体为：

```
movq      (%rdi), %rax
cvtsi2sdq %rax, %xmm2
addsd     %xmm0, %xmm2    # 先做 (double)*p + x
cvtss2sd  %xmm1, %xmm1
addsd     %xmm1, %xmm2    # 再加 (double)y
movapd    %xmm2, %xmm0
ret
```

浮点加法不普遍满足结合律，所以不能随意先算x+y再加*p。此题强调顺序与通道，不要求指令必须与这里逐字相同。

按错误回看

- P1错：回图4–4，标低地址字节权重。
- P2、P7错：先写完整指针类型，再确定加一的步长。
- P3错：把整数/指针与浮点参数分别排队。
- P4错：写清“字符数n、总写入n+1、最后地址B+n”。
- P5、P6错：区分取指令和从栈读取返回地址。
- P8错：每放一个成员先对齐，最后再做整体对齐；反推时保留所有满足条件的解。

第5讲纠错题答案

ICS 第一次小测复习 · 独立练习答案 · 请先完成题目再核对

对应第5讲的新题库。这些都是助手补充题。选择实际薄弱的两组验证，每题都要求结果与关键理由正确；不另虚构考试分值。

A 整数与类型

A1：0x92无符号为146；8位补码值为146−256=−110；扩展成16位为0xFF92。符号扩展保留有符号数值，不是保留无符号解释。

A2：x转为32位unsigned，结果为$2^{32}-3=4294967293$，所以x<u为假。比较不是在数学整数−3与2之间直接进行。

B 浮点

B1：`0 010 1000`阶码非零且非全1，是规格化数，E=2−3=−1，M=1.5，数值0.75。`0 000 0010`是非规格化数，E=1−3=−2，M=2/16=1/8，数值1/32。后一种没有隐藏的1。

B2：保留1.101，舍弃100，正好中点。保留的最低位是1，进位后为 **1.110**，最低位为0。不是总向下舍入，也不是把小数前三位101解释为十进制101后判断整体数值的奇偶。

C 寻址与寄存器

C1：0x18+0x200+5×8=0x240。lea只计算这个数，不读0x240处的内存。

C2：写ax只替换低16位，结果 **0xFFFFFFFF1234ABCD**；写eax则清高32位，结果 **0x000000000000ABCD**。两问均从原值出发，不连续执行。

D 控制流

D1：cmp计算rdi−rsi，setb表示无符号rdi<rsi。有符号小于改用 **setl**。不能把b与l理解为仅有拼写差异。

D2：对应的示意C为：

```c
unsigned long sum(unsigned long n) {
    unsigned long r=0;
    while (n!=0) {
        r+=n;
        n-=1;
    }
    return r;
}
```

n=4返回10，n=0返回0。这是先判断后执行的循环；若大n使和超出64位，则按无符号模2^{64}处理，不能无条件宣称数学和仍可精确表示。

E 调用栈

E1：call后rsp=0x6F8；push后0x6F0；减16后 **0x6E0**。返回地址保存位置仍为0x6F8，所以相对当前rsp偏移 **0x18，即24字节**。保存的值是0x401025。

E2：可以依赖遵循ABI的被调用函数在返回时恢复rbx。它在执行中可以临时修改，不是硬件禁止写。本函数也必须为自己的调用者保留进入时的rbx，通常先保存，退出前恢复。角色是相对的：同一函数既可能是调用者，也可能是被调用者。

F 数据布局

F1：一行六个short，偏移2×(3×6+2)=40=0x28，地址 **0x5028**。行号乘列数6。

F2：c偏移0，d偏移8，i偏移16；成员结束于20，整体对齐8，所以总大小 **24**。最后四字节是尾部填充。

G 联合与内存安全

G1：小端结果 **0xDDCCBBAA**。低地址AA是低八位，其余字节有不同的位权；不是AA+BB+CC+DD。

G2：不可执行栈不会阻止越界写坏局部变量、指针或其他数据；若控制数据被破坏，还可能返回到原本就可执行的代码。NX限制在哪里取指令，不负责检查每次数组写入是否合法，所以仍需要长度检查等措施。